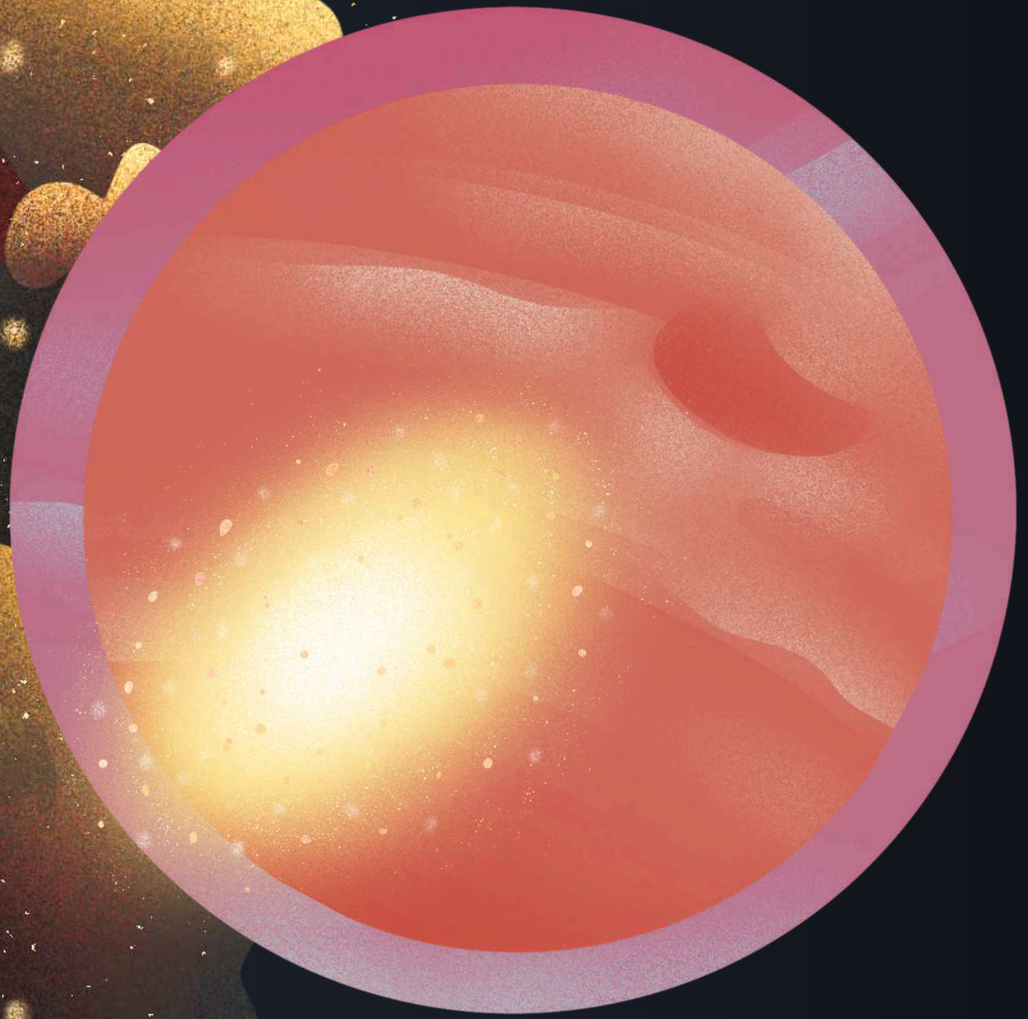
MISIÓN EN MARTE

Bances Panaque, Jose Junior
Barzola Reategui, Josué Obed
Cerdan Dongo, Rebeca Milagros
Salcedo Siguas, Yanella Stefhany



RESUMEN

- El jugador controla a un explorador en Marte.
- Debe recolectar minerales, contruir bases, interactuar con marcianos y sobrevivir a amenazas.
  - **Objetivo narrativo:** *Conseguir escapar del planeta.*

Nivel 1 – Supervivencia
Recolectar recursos y sobrevivir a la radiación.

Nivel 2 – Defensa
Proteger las bases frente a los meteoritos y amenazas.

Nivel 3 – Escape
Alcanzar el objetivo final y escapar de Marte.

Objetivo y herramientas de desarrollo

Objetivo principal:

Demostrar la aplicación de los fundamentos de POO mediante el diseño e implementación de un videojuego funcional.



PYTHON 3:
Lenguaje, módulos abc y typing



PYGAME:
Ventana, Game Loop y teclado



DRAW.IO:
Modelado UML.



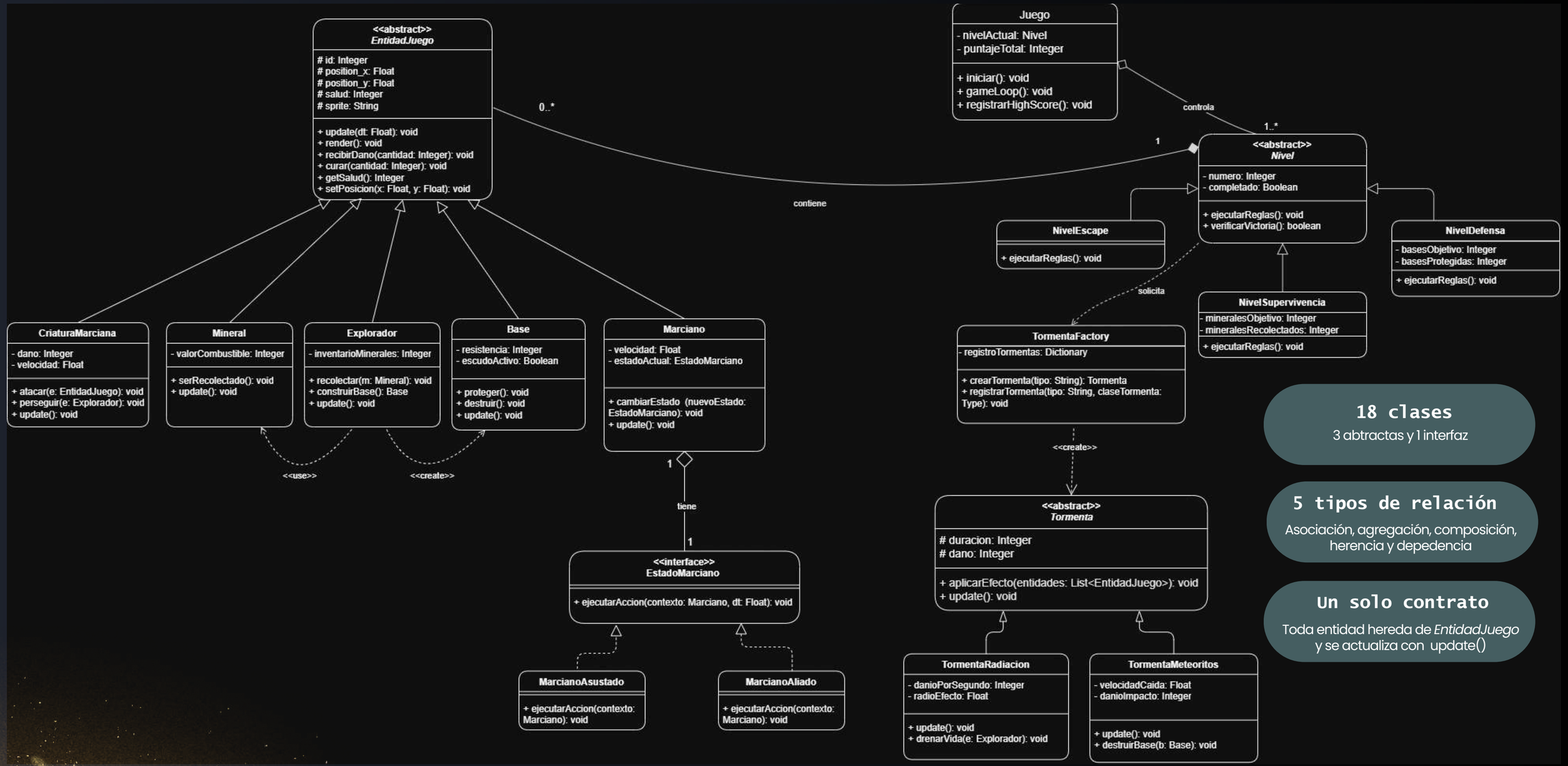
VISUAL STUDIO CODE:
Edición y ejecución del juego



GITHUB:
Repositorio POO_Group_03 del equipo

```
POO_Group_03/
├── main.py
├── requirements.txt
├── assets/
├── docs/
└── src/
    ├── entidades/
    ├── niveles/
    ├── tormentas/
    ├── juego.py
    └── config.py
```

DIAGRAMA UML



18 clases
3 abtractas y 1 interfaz

5 tipos de relación
Asociación, agregación, composición, herencia y depedencia

Un solo contrato
Toda entidad hereda de *EntidadJuego* y se actualiza con update()

CLASES PRINCIPALES

Juego	**Controla el bucle principal, las fases y el HUD. Coordina la ejecución del videojuego.**
EntidadJuego	**Define el comportamiento común de las entidades. Permite reutilizar atributos y métodos mediante herencia.**
Explorador	**Representa al personaje controlado por el jugador. Gestiona el movimiento, la recolección y la construcción de bases.**
Nivel	**Organiza entidades, tormentas y reglas. Centraliza la lógica de cada etapa.**
Mineral	**Representa los recursos recolectables. Permite gestionar el combustible y los recursos del juego.**
Base	**Funciona como refugio para el explorador. Incorpora protección y curación.**
Marciano	**Representa un habitante con comportamiento variable. Utiliza el patrón State.**
TormentaFactory	**Crea las tormentas registradas. Aplica el patrón Factory Method.**

ENCAPSULAMIENTO Y ABSTRACCIÓN



Atributos críticos privados

manipulación de nombres: __salud, __estadoActual



Sin setters destructivos

solo recibirDano() y curar() cambian la salud



Abstracción con ABC

EntidadJuego, Nivel, Tormenta y EstadoMarciano

```python
def recibirDano(self, cantidad: int) -> None:
    if cantidad < 0:
        raise ValueError("el daño no puede ser negativo")
    if not self.estaViva():
        return
    self.__salud = max(0, self.__salud - cantidad)
    if self.__salud == 0:
        self._alMorir()

def curar(self, cantidad: int) -> None:
    if cantidad < 0:
        raise ValueError("la cura no puede ser negativa")
    if self.estaViva():
        self.__salud = min(self.__saludMaxima, self.__salud + cantidad)
```

RELACIONES ENTRE OBJETOS

Asociación

Un objeto conoce a otro para colaborar.

En el juego:

Base → Explorador (su dueño)
CriaturaMarciana → Explorador

Agregación

El todo recibe partes que viven por separado.

En el juego:

Juego ◇– Nivel
Marciano ◇–EstadoMarciano

Composición

Las partes nacen y mueren con el todo.

En el juego:

Nivel ◆– Entidades
Nivel ◆–Tormentas

RELACIONES ENTRE OBJETOS

Herencia

Las clases derivadas comparten una estructura y un contrato común.

En el juego:

EntidadJuego ⇐ Explorador, Mineral, Base, Marciano, CriaturaMarciana

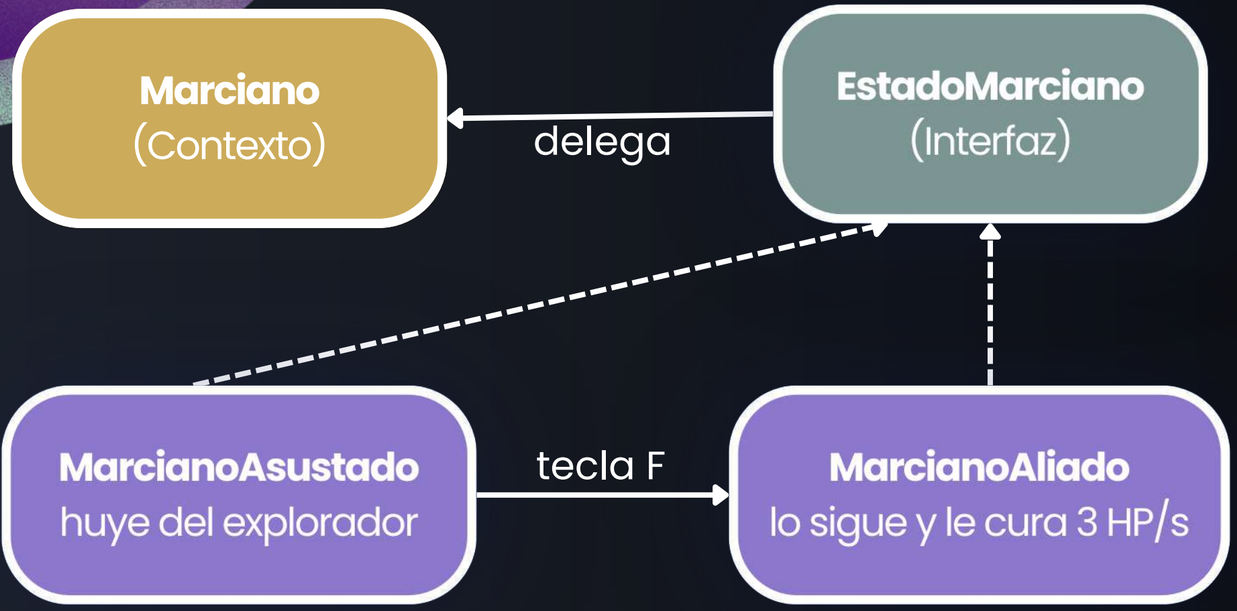
Dependencia

El nivel utiliza la fábrica para solicitar la creación de tormentas.

En el juego:

Nivel ⇢ TormentaFactory

PATRÓN STATE:
EL MARCIANO




```
self.__estadoActual
    .ejecutarAccion(
    self, dt)
```

Sin este patrón tendríamos muchos if dentro de la clase Marciano.

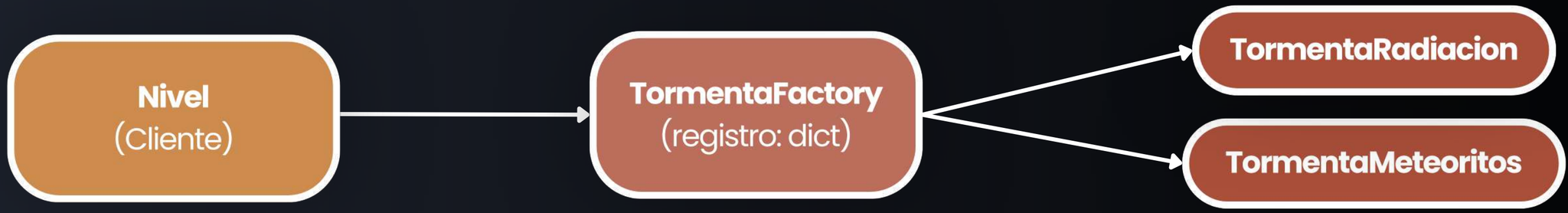
HERENCIA Y POLIMORFISMO



update().render()

```
for entidad in self.__entidades:
    entidad.update(dt)
for t in self.__tormentas:
    t.update(dt)
    t.aplicarEfecto(
       self.__entidades)
```

PATRÓN FACTORY METHOD



crearTormenta(“radiacion”)

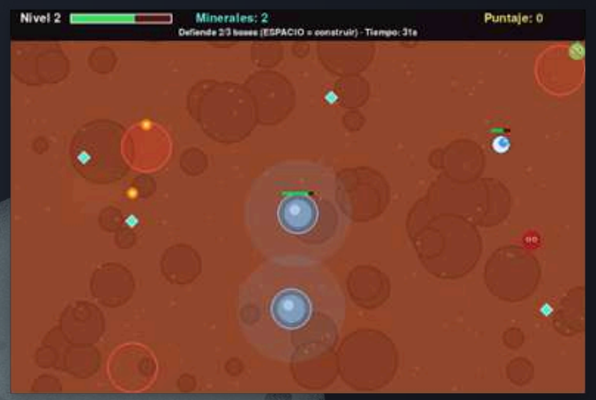
```
fabrica = TormentaFactory()
fabrica.registrarTormenta(“radiacion”, TormentaRadiacion)
fabrica.registrarTormenta(“meteoritos”, TormentaMeteoritos)
```

Open/Closed
Tormenta nueva = 1 línea, sin tocar la fábrica

EL JUEGO
EN ACCIÓN









Demo en vivo

- Menú y controles
- Nivel 1: recolectar combustible
- Nivel 2: defender 3 bases por 40 s
- Nivel 3: llegar al cohete

Controles

WASD/↑ ←↓→ = mover
ESPACIO = base
F = aliado
ESC = salir

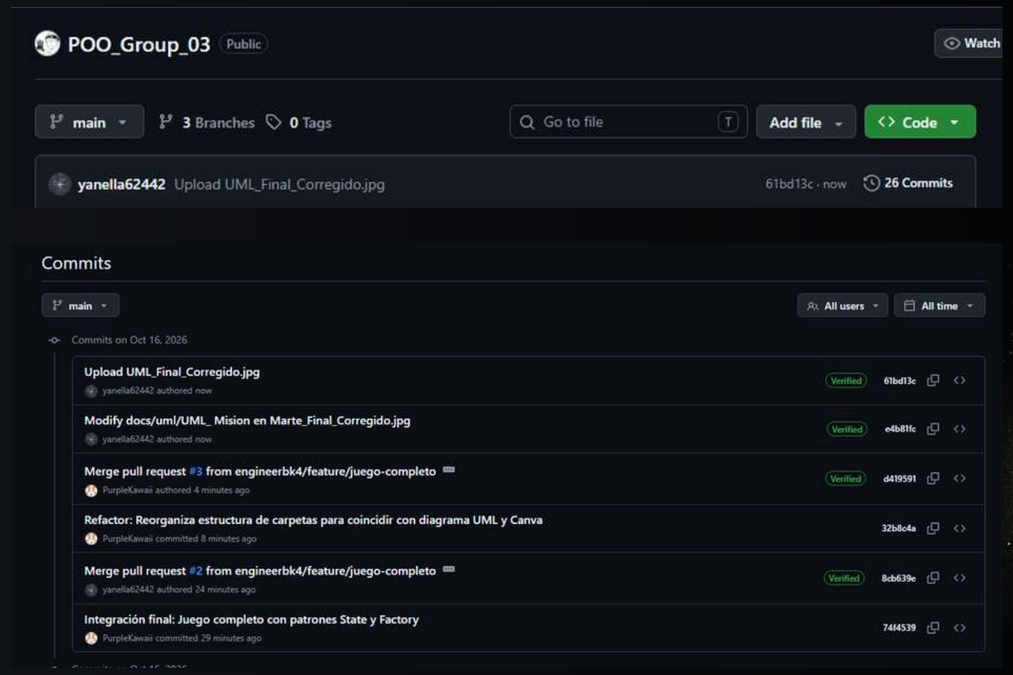
TRABAJO EN EQUIPO CON GIT

Rama propia
feature/<tarea> por cada integrante

Commits pequeños
Mensajes claros y en presente

Pull Request
Revisión cruzada de un compañero

Merge a main
main siempre contiene código que funciona



Repositorio:
POO_Group_03

CONCLUSIONES Y SIGUIENTES PASOS

Logros

- 18 clases y los 6 pilares de POO aplicadas.
- Juego funcional: 3 niveles, enemigos y tormentas.
- Patrones Factory Method y State.

Siguientes pasos

- APIClient para la tabla de puntuaciones.
- Sprites y sonido.
- Balanceo y más niveles.


MUCHAS
GRACIAS